

Empirical Analysis of the Capital Asset Pricing Model (CAPM) - MIT 18.642

Table of contents

1	Capital Asset Pricing Model Theory	3
2	Install and Load Packages	3
3	Load Data Set (if available)	5
4	Construct Data Set if it is not available	5
4.1	Collect data for S&P 500 Index and 4 stocks	5
4.2	Plot chart for the S&P 500 Index	6
4.3	Collect yields on U.S. Treasury bonds and crude oil prices	7
4.4	Plot U.S. Treasury yield series	8
4.5	Plot crude oil price series	9
4.6	Combine stock and macro-economic data	9
4.7	Process missing values	10
4.8	Save data set	11
5	Historical Financial Data	11
5.1	Plot GE stock prices, market-portfolio proxy (S&P 500) and risk-free rate (3-month treasury yield)	13
5.2	Compute number of days between consecutive observations	16
5.3	Compute daily returns and log-returns	18
5.4	Compute daily log-returns of the risk-free rate proxy: 3-Month Treasury yield .	18
5.5	Compute excess returns over risk free rate proxy	19
5.6	Save computed values	20
6	Fitting the Linear Regression for CAPM	20
7	R-squared: Proportion of Variance	22

8	Residuals Analysis	23
9	Regression Diagnostics: Influence measures	28
10	Adding Macro-economic Factors to CAPM	33
10.1	Linear regression for the simple CAPM	33
10.2	Linear regression for the extended CAPM	34
11	Analysis of Variance	35
11.1	ANOVA Comparison of Nested Linear Regression Models	36
12	Exxon-Mobil stock	38

1 Capital Asset Pricing Model Theory

Sharpe (1964) and Lintner (1965) developed the Capital Asset Pricing Model for a market in which investors have the same expectations, hold portfolios of risky assets that are mean-variance efficient, and can borrow and lend money freely at the same risk-free rate. In such a market, the expected return of asset j is

$$E[R_j] = R_{riskfree} + \beta_j(E[R_{Market}] - R_{riskfree})$$

$$\beta_j = Cov[R_j, R_{Market}] / Var[R_{Market}]$$

where R_{Market} is the return on the market portfolio and $R_{riskfree}$ is the return on the risk-free asset.

Consider fitting the simple linear regression model of a stock's daily excess return on the market-portfolio daily excess return, using the S&P 500 Index as the proxy for the market return and the 3-month Treasury constant maturity rate as the risk-free rate. The linear model is given by:

$$R_{j,t}^* = \alpha_j + \beta_j R_{Market,t} + \varepsilon_{j,t}, \quad t=1,2, \dots$$

where $\varepsilon_{j,t}$ are white noise $WN(0, \sigma^2)$.

Under the assumptions of the CAPM, the regression parameters (α_j, β_j) are such that β_j is the same as in the CAPM model, and α_j is zero.

2 Install and Load Packages

```
# Install/load R packages
# Collect historical financial data from internet
# Create time series data matrix: casestudy1.data0.0
#   Closing prices on stocks (BAC, GE, JDSU, XOM)
#   Closing values of indexes (SP500)
#   Yields on constant maturity US rates/bonds (3MO, 1YR, 5YR, 10 YR)
#   Closing price on crude oil spot price

# 0. Install and load packages ----
#
```

```

# 0.1 Install packages ---
# Load packages into R session (and install if necessary)

# require() returns TRUE if the package is successfully loaded
# and FALSE if it is not found.
if (!require("pacman")) install.packages("pacman")
if (!require("conflicted")){
  # Install specific version from GitHub if not already present
  devtools::install_github("r-lib/conflicted")
}

# Note: p_load checks for existence before attempting installation
pacman::p_load(
  quantmod, # Financial quantitative modeling
  tseries,  # Time series analysis
  vars,     # Vector Autoregressive models
  fxregime, # Exchange rate regimes
  moments,
  rugarch,
  xts,
  graphics,
  tidyquant,
  tidyverse,
  fpp2
)

```

```

# 0.2 Load packages into R session

```

```

library("quantmod")
library("tseries")
library("vars")
library("fxregime")
library("moments")
library("rugarch")
library("graphics")
library("tidyquant")
library("tidyverse")
library("fpp2")
library("ggplot2")

```

3 Load Data Set (if available)

```
file_name="mit18_642_f24_lec06_1_3 - Empirical Analysis of CAPM"

# Check whether the .RData file exists in the working directory
dataset_exists=file.exists(file_name)

if (dataset_exists) {
  # Load objects stored in the .RData file
  load(file_name)
} else {

  # Display a message if the file cannot be found
  message("File not found")
}
```

4 Construct Data Set if it is not available

4.1 Collect data for S&P 500 Index and 4 stocks

```
if (!dataset_exists) {

# 1. Load data into R session ----

# 1.1 Stock Price Data from Yahoo

# 1.1.1 Set start and end date for collection in YYYYMMDD (numeric) format

date_start <- "2000-01-01"
date_end <- "2013-05-31"

# 1.1.2 Collect historical data for S&P 500 Index and 4 stocks

# Stock tickers

# ^GSPC = S&P 500 Index
# BAC = Bank of America
```

```

# GE      = General Electric
# VIAV    = Viavi Solutions
# XOM     = Exxon Mobil
stocks_tickers <- c("BAC", "GE", "VIAV", "XOM", "^GSPC")

# Download stock data
stock_data <- tq_get(
  stocks_tickers,
  get = "stock.prices",
  from = date_start,
  to = date_end
)

# Replace the ticker "^GSPC" with a cleaner one: "SP500"
stock_data$symbol[stock_data$symbol == "^GSPC"] <- "SP500"
}

```

4.2 Plot chart for the S&P 500 Index

```

if (!dataset_exists) {

# Plot OHLCV chart for the S&P 500 index

# Filter S&P 500 data from the dataset
SP500 <- stock_data %>%
  dplyr::filter(symbol == "SP500")

# Convert tibble to xts object using OHLCV columns
SP500 <- xts(
  x = SP500[, c("open", "high", "low", "close", "volume")],
  order.by = SP500$date
)

# Plot OHLCV chart
chartSeries(SP500)
}

```

4.3 Collect yields on U.S. Treasury bonds and crude oil prices

```
if (!dataset_exists) {

# 1.2 Federal Reserve Economic Data (FRED) from the St. Louis Federal Reserve

# Series name| Description

# DGS3MO      | 3-Month Treasury, constant maturity rate
# DGS1        | 1-Year Treasury, constant maturity rate
# DGS5        | 5-Year Treasury, constant maturity rate
# DGS10       | 10-Year Treasury, constant maturity rate

# DAAA        | Moody's Seasoned Aaa Corporate Bond Yield
# DBAA        | Moody's Seasoned Baa Corporate Bond Yield

# DCOILWTICO  | Crude Oil Prices: West Texas Intermediate (WTI) - Oklahoma

fred_series <- c(
  "DGS3MO",
  "DGS1",
  "DGS5",
  "DGS10",
  "DAAA",
  "DBAA",
  "DCOILWTICO"
)

# Apply tidyquant function
#   tq_get(series_name, get = "economic.data")
#       Returns historical data for any symbol at the website
#       http://research.stlouisfed.org/

# Download Federal Reserve Economic Data (FRED)
fred_data <- tq_get(
  fred_series,
  get = "economic.data",
  from = date_start,
  to = date_end
)

# Determine data dimensions
```

```

dim(fred_data)

# Check object class
class(fred_data)
}

if (!dataset_exists) {

# Convert from long to wide format
macroeconomic_data <- tidyr::pivot_wider(
  fred_data,
  names_from = symbol,
  values_from = price
)

# Determine data dimensions
dim(macroeconomic_data)

# Check object class
class(macroeconomic_data)

# Count missing values in each column
colSums(is.na(macroeconomic_data))
}

```

4.4 Plot U.S. Treasury yield series

```

if (!dataset_exists) {

# Plot the interest rates series together

ts.plot(
  as.ts(
    dplyr::select(macroeconomic_data, -date, -DCOILWTICO)
  ),
  col = rainbow(6),
  main = "FRED Data: Rates"
)

# Add legend to plot

```



```
graphics::legend(
  "topright",
  legend=colnames(macroeconomic_data)[2:7],
  lty=rep(1,6),
  col=rainbow(6),
  cex=0.75
)
}
```

4.5 Plot crude oil price series

```
if (!dataset_exists) {
# Plot the Crude Oil Price

# Convert to xts
oil_prices <- xts(
  x = macroeconomic_data$DCOILWTICO,
  order.by = macroeconomic_data$date
)

chartSeries(
  oil_prices,
  name = "Crude Oil Price (WTI)"
)
}
```

4.6 Combine stock and macro-economic data

```
if (!dataset_exists) {
# 2.2 Merge the closing prices for the stock market data series

# Keep closing prices and reshape from long to wide format
stock_closing_prices <- stock_data %>%
  dplyr::select(date, symbol, close) %>%
  tidyr::pivot_wider(
    names_from = symbol,
    values_from = close
  )
}
```

```

# Sort data by date in ascending order
stock_closing_prices <- dplyr::arrange(stock_closing_prices, date)

# 2.3 Merge the yahoo and Fred data together

# 2.3.1 Merge preserving all dates

# Outer join to compare missing alignment across datasets
final_dataset <- full_join(
  stock_closing_prices,
  macroeconomic_data,
  by = "date"
)

# Check total rows after full merge
dim(final_dataset)
}

```

4.7 Process missing values

```

if (!dataset_exists) {
# Count missing values per column
colSums(is.na(final_dataset))
}

```

Delete observations where S&P 500 is missing

```

if (!dataset_exists) {
# 2.3.2 Delete observations where S&P 500 is missing

final_dataset <- final_dataset %>%
  dplyr::filter(!is.na(SP500))

# Check total rows after deleting missing values
dim(final_dataset)

# Count missing values per column
colSums(is.na(final_dataset))
}

```

Fill missing values using last observation carried forward (LOCF)

```
if (!dataset_exists) {  
  # Remaining missing values are in interest rates and crude oil prices  
  
  # This occurs because equity markets and bond/commodity markets  
  # have different holidays/trading calendars  
  
  # Fill missing values using last observation carried forward (LOCF)  
  final_dataset <- na.locf(final_dataset)  
  
  # Count missing values per column  
  colSums(is.na(final_dataset))  
}
```

4.8 Save data set

```
if (!dataset_exists) {  
  
  # Vector containing object names to save  
  objects_to_save <- c(  
    "date_end",  
    "date_start",  
    "final_dataset"  
  )  
  
  # Save selected objects  
  save(  
    list = objects_to_save,  
    file = file_name  
  )  
}
```

5 Historical Financial Data

```
dim(final_dataset)
```

```
[1] 3371 33
```

```
names(final_dataset)
```

```
[1] "date"          "BAC"
[3] "GE"            "VIAV"
[5] "XOM"           "SP500"
[7] "DGS3MO"        "DGS1"
[9] "DGS5"          "DGS10"
[11] "DAAA"          "DBAA"
[13] "DCOILWTICO"    "days_elapsed"
[15] "return_BAC"    "return_GE"
[17] "return_VIAV"   "return_XOM"
[19] "return_SP500"  "diff_log_price_BAC"
[21] "diff_log_price_GE" "diff_log_price_VIAV"
[23] "diff_log_price_XOM" "diff_log_price_SP500"
[25] "diff_log_DGS3MO_risk_free" "excess_BAC"
[27] "excess_GE"     "excess_VIAV"
[29] "excess_XOM"    "excess_SP500"
[31] "return_DCOILWTICO" "diff_log_price_DCOILWTICO"
[33] "excess_DCOILWTICO"
```

```
str(final_dataset)
```

```
tibble [3,371 x 33] (S3: tbl_df/tbl/data.frame)
```

```
$ date          : Date[1:3371], format: "2000-01-04" "2000-01-05" ...
$ BAC           : num [1:3371] 22.8 23 25 24.3 23.5 ...
$ GE            : num [1:3371] 230 230 233 242 242 ...
$ VIAV          : num [1:3371] 389 360 341 409 456 ...
$ XOM           : num [1:3371] 38.4 40.5 42.6 42.5 41.9 ...
$ SP500         : num [1:3371] 1399 1402 1403 1441 1458 ...
$ DGS3MO        : num [1:3371] 5.43 5.44 5.41 5.38 5.42 5.43 5.45 5.41 5.41 5.56
$ DGS1          : num [1:3371] 6 6.05 6.03 6 6.07 6.13 6.16 6.1 6.13 6.15 ...
$ DGS5          : num [1:3371] 6.4 6.51 6.46 6.42 6.49 6.57 6.63 6.54 6.59 6.65
$ DGS10         : num [1:3371] 6.49 6.62 6.57 6.52 6.57 6.67 6.72 6.63 6.69 6.75
$ DAAA          : num [1:3371] 7.69 7.78 7.72 7.69 7.72 7.81 7.85 7.8 7.84 7.9 .
$ DBAA          : num [1:3371] 8.21 8.29 8.24 8.22 8.27 8.36 8.4 8.38 8.42 8.47
$ DCOILWTICO    : num [1:3371] 25.6 24.6 24.8 24.8 24.7 ...
$ days_elapsed  : num [1:3371] 1 1 1 1 3 1 1 1 1 4 ...
$ return_BAC    : num [1:3371] NA 0.011 0.0855 -0.0262 -0.0347 ...
```

```

$ return_GE : num [1:3371] NA -0.001736 0.01337 0.038721 -
0.000413 ...
$ return_VIAV : num [1:3371] NA -0.0752 -0.0537 0.2016 0.1136 ...
$ return_XOM : num [1:3371] NA 0.05452 0.0517 -0.00293 -
0.01398 ...
$ return_SP500 : num [1:3371] NA 0.001922 0.000956 0.02709 0.01119 ...
$ diff_log_price_BAC : num [1:3371] NA 0.0109 0.082 -0.0266 -0.0353 ...
$ diff_log_price_GE : num [1:3371] NA -0.001738 0.013281 0.037991 -
0.000413 ...
$ diff_log_price_VIAV : num [1:3371] NA -0.0782 -0.0552 0.1836 0.1076 ...
$ diff_log_price_XOM : num [1:3371] NA 0.05308 0.05041 -0.00294 -
0.01408 ...
$ diff_log_price_SP500 : num [1:3371] NA 0.00192 0.000955 0.02673 0.011128 ...
$ diff_log_DGS3MO_risk_free: num [1:3371] 0.000151 0.000151 0.00015 0.000149 0.000452 ...
$ excess_BAC : num [1:3371] NA 0.0108 0.0819 -0.0268 -0.0357 ...
$ excess_GE : num [1:3371] NA -0.001889 0.013131 0.037841 -
0.000865 ...
$ excess_VIAV : num [1:3371] NA -0.0784 -0.0554 0.1835 0.1071 ...
$ excess_XOM : num [1:3371] NA 0.05293 0.05026 -0.00309 -
0.01453 ...
$ excess_SP500 : num [1:3371] NA 0.001769 0.000805 0.026581 0.010676 ...
$ return_DCOILWTICO : num [1:3371] NA -0.0356 0.00568 0 -0.00323 ...
$ diff_log_price_DCOILWTICO: num [1:3371] NA -0.03625 0.00566 0 -0.00323 ...
$ excess_DCOILWTICO : num [1:3371] NA -0.036403 0.005513 -0.000149 -
0.003684 ...
- attr(*, "na.action")= 'omit' Named int 1
..- attr(*, "names")= chr "1"

```

5.1 Plot GE stock prices, market-portfolio proxy (S&P 500) and risk-free rate (3-month treasury yield)

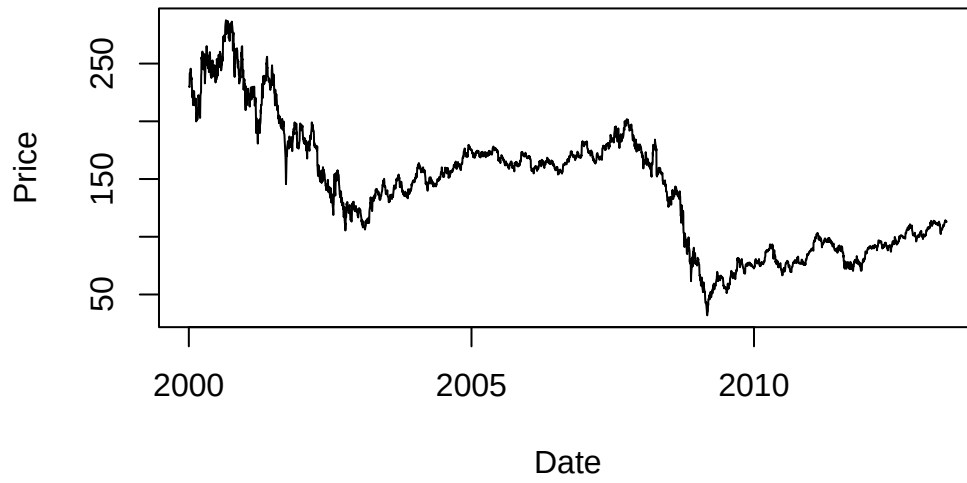
We first plot the raw data for the stock *GE*, the market-portfolio index *SP500*, and the risk-free interest rate.

```

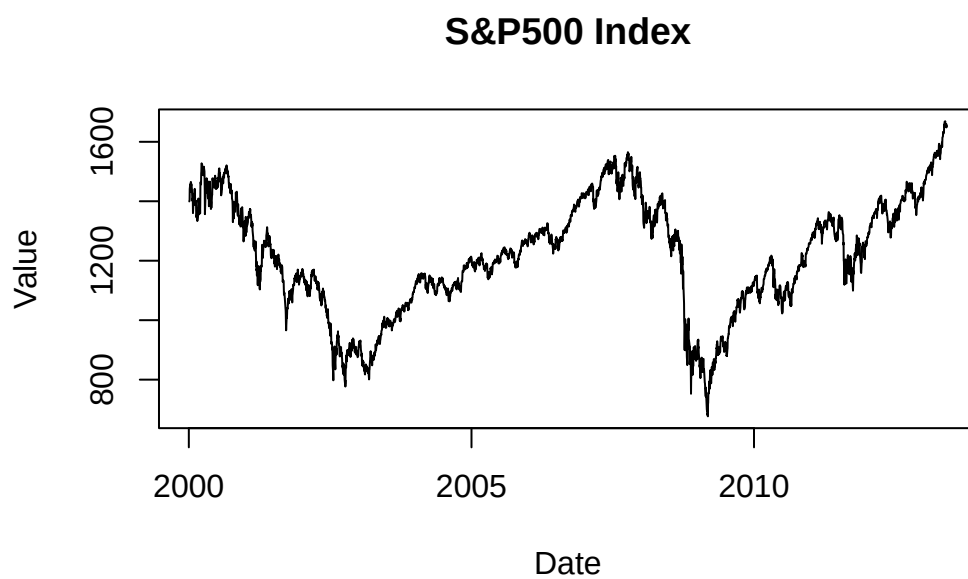
plot(final_dataset$date, final_dataset$GE,
     type = "l",
     xlab = "Date",
     ylab = "Price",
     main = "GE Stock"
)

```

GE Stock

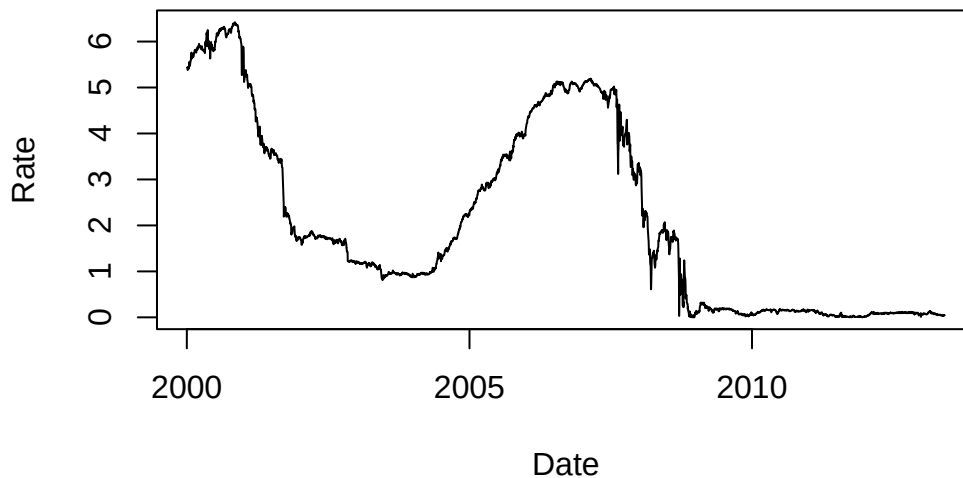


```
plot(final_dataset$date, final_dataset$SP500,  
      type = "l",  
      xlab = "Date",  
      ylab = "Value",  
      main = "S&P500 Index")
```



```
plot(final_dataset$date, final_dataset$DGS3M0,  
      type = "l",  
      xlab = "Date",  
      ylab = "Rate",  
      main = "3-Month Treasury Rate (Constant Maturity)")
```

3-Month Treasury Rate (Constant Maturity)



Now we construct the variables with the log daily returns of GE and the SP500 index as well as the risk-free asset returns.

```
# Define bond/debt tickers
bonds_tickers <- c("DGS3MO", "DGS1", "DGS5", "DGS10", "DAAA", "DBAA")

# Define stocks tickers
stocks_tickers <- c("BAC", "GE", "VIAV", "XOM", "SP500", "DCOILWTICO")
```

5.2 Compute number of days between consecutive observations

```
# Ensure time ordering
final_dataset <- final_dataset %>% arrange(date)

# Compute number of days between consecutive observations
final_dataset <- final_dataset %>%
  arrange(date) %>%
  mutate(
    # Number of days between consecutive observations
    days_elapsed = c(1, as.numeric(diff(date)))
  )
```

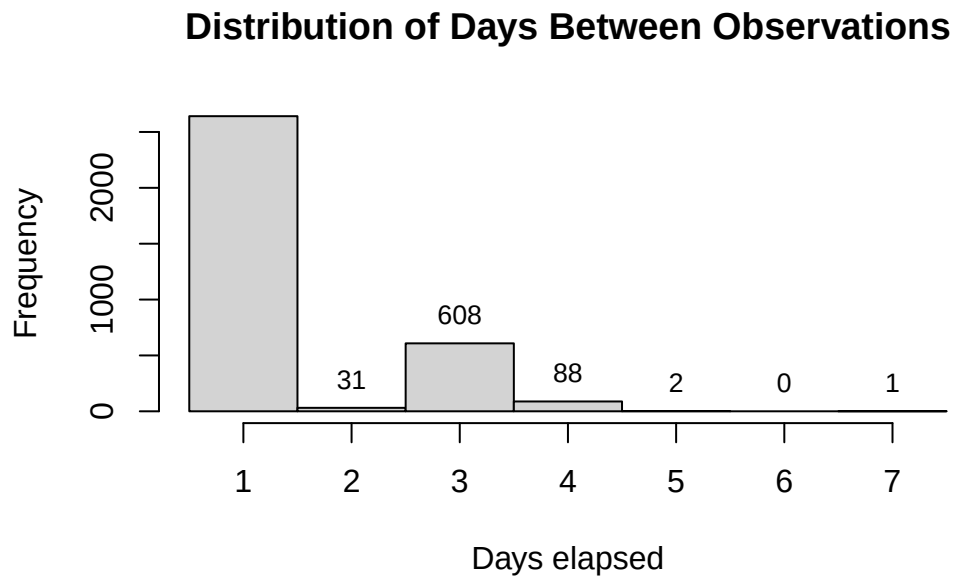


```

# Compute histogram without plotting labels manually first
h <- hist(
  final_dataset$days_elapsed,
  breaks = seq(
    floor(min(final_dataset$days_elapsed)) - 0.5,
    ceiling(max(final_dataset$days_elapsed)) + 0.5,
    by = 1
  ),
  main = "Distribution of Days Between Observations",
  xlab = "Days elapsed",
  ylab = "Frequency"
)

# Add frequency labels on top of bars
text(
  x = h$mids,
  y = h$counts,
  labels = h$counts,
  pos = 3,          # above the bar
  cex = 0.8,        # text size
)

```



```
rm(h)
```

5.3 Compute daily returns and log-returns

```
# Compute simple returns for selected tickers
final_dataset <- final_dataset %>%
  arrange(date) %>%
  mutate(across(stocks_tickers,
    ~ c(NA, diff(.) / head(., -1)), # simple returns
    .names = "return_{.col}"
  ))
```

```
Warning: There was 1 warning in `mutate()`.
i In argument: `across(stocks_tickers, ~c(NA, diff(.) / head(., -1)), .names =
  "return_{.col}")`.
Caused by warning:
! Using an external vector in selections was deprecated in tidysselect 1.1.0.
i Please use `all_of()` or `any_of()` instead.
# Was:
data %>% select(stocks_tickers)

# Now:
data %>% select(all_of(stocks_tickers))
```

See <<https://tidysselect.r-lib.org/reference/faq-external-vector.html>>.

```
# Compute log returns for selected tickers
final_dataset <- final_dataset %>%
  mutate(across(all_of(stocks_tickers),
    ~ c(NA, diff(log(.))), # diff log prices = log returns
    .names = "diff_log_price_{.col}"
  ))
```

5.4 Compute daily log-returns of the risk-free rate proxy: 3-Month Treasury yield

If B_t is the price of a bond at time t , then we have that:

$$\Delta \log B_t = \log(B_t) - \log(B_{t-1}) = \log\left(\frac{B_t}{B_{t-1}}\right) = \log\left(1 + \frac{B_t}{B_{t-1}} - 1\right) = \log(1 + R_t)$$

where the simple return R_t is equal to $R_t = \frac{B_t - B_{t-1}}{B_{t-1}}$.

```
# Compute difference between log prices of the risk-free asset
# (3-Month Treasury Rate (Constant Maturity))
# Adjusted using elapsed days and standard of 360 days per year

final_dataset <- final_dataset %>%
  mutate(
    diff_log_DGS3MO_risk_free = log(1 + 0.01 * DGS3MO * days_elapsed / 360)
  )
```

5.5 Compute excess returns over risk free rate proxy

```
# Compute excess returns over risk-free rate using a loop
for (ticker in stocks_tickers) {

  # Construct column name for log returns of each stock
  log_ret_col <- paste0("diff_log_price_", ticker)

  # Construct output column name
  excess_col <- paste0("excess_", ticker)

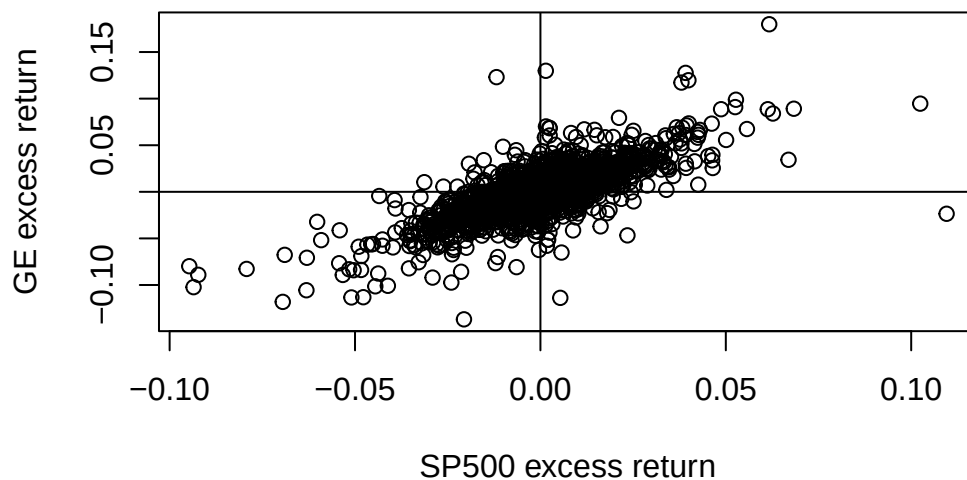
  # Compute excess return
  final_dataset[[excess_col]] <-
    final_dataset[[log_ret_col]] - final_dataset$diff_log_DGS3MO_risk_free
}
rm(ticker, log_ret_col, excess_col)
```

Now we plot the excess returns of GE vs those of the SP500:

```
# Scatter plot of excess returns
plot(final_dataset$excess_SP500,
     final_dataset$excess_GE,
     xlab = "SP500 excess return",
     ylab = "GE excess return",
     main = "GE excess return vs SP500 excess return")

abline(h=0, v=0)
```

GE excess return vs SP500 excess return



5.6 Save computed values

```
# Vector containing object names to save
objects_to_save <- c(
  "date_end",
  "date_start",
  "final_dataset"
)

# Save selected objects
save(
  list = objects_to_save,
  file = file_name
)
```

6 Fitting the Linear Regression for CAPM

The linear regression model is fit using the R-function `lm( )`:

```
# Fit linear model
linear_regression <- lm(excess_GE ~ excess_SP500, data = final_dataset)

names(linear_regression) #element names of list object model
```

```
[1] "coefficients" "residuals"      "effects"      "rank"
[5] "fitted.values" "assign"         "qr"           "df.residual"
[9] "na.action"     "xlevels"        "call"         "terms"
[13] "model"
```

```
summary(linear_regression) #function summarizing objects created by lm()
```

Call:

```
lm(formula = excess_GE ~ excess_SP500, data = final_dataset)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.153047	-0.005734	-0.000304	0.005575	0.137353

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.0002540	0.0002385	-1.065	0.287
excess_SP500	1.1843741	0.0178781	66.247	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.01385 on 3368 degrees of freedom

(1 observation deleted due to missingness)

Multiple R-squared: 0.5658, Adjusted R-squared: 0.5657

F-statistic: 4389 on 1 and 3368 DF, p-value: < 2.2e-16

Note that the t -statistic for the intercept α_{GE} is not significant:

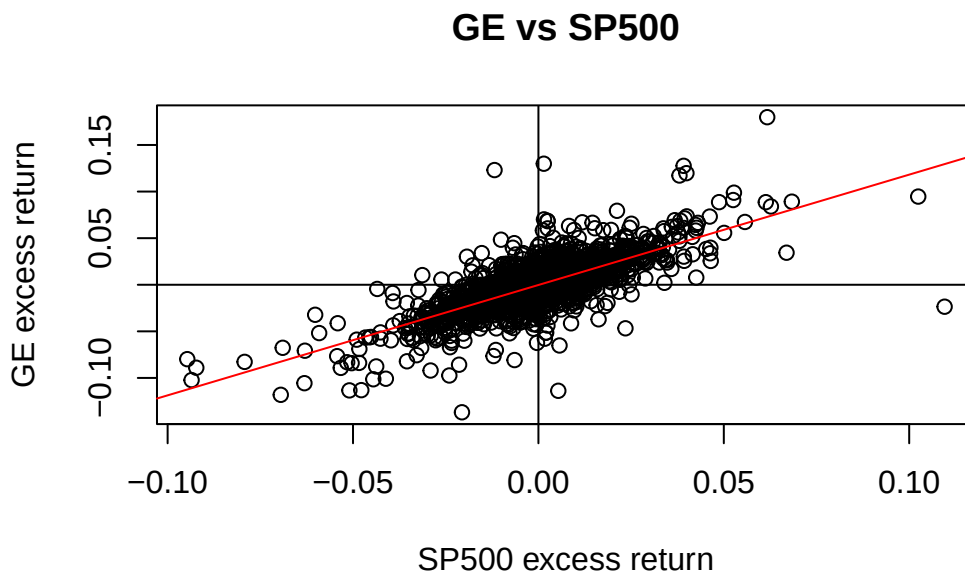
```
linear_regression.summary <- summary(linear_regression)
tstat.intercept <- round(
  linear_regression.summary$coefficients["(Intercept)", "t value"],
  digits=4)
print(tstat.intercept)
```

```
[1] -1.0649
```

```
# Scatter plot of excess returns
plot(final_dataset$excess_SP500,
     final_dataset$excess_GE,
     xlab = "SP500 excess return",
     ylab = "GE excess return",
     main = "GE vs SP500")

abline(h=0,v=0)

# Add regression line
abline(linear_regression, col = "red")
```



7 R-squared: Proportion of Variance

The R-squared statistic is the proportion of the variance of the dependent variable which is explained by the linear regression model. This statistic is most easily interpreted by displaying the plot of the Actual values versus Fitted values for the regression.

8 Residuals Analysis

When fitting a linear regression model the ideal assumptions are that the regression errors are independent and identically distributed Gaussian random variables.

In this case, the t-distribution applies for testing the statistical significance of coefficient estimates and the F-distribution applies for testing the statistical significance of the overall regression model.

Typically, residuals analysis begins by evaluating the consistency of the data with the assumption of Gaussian errors.

We plot the histogram of the model residuals:

```
residuals <- as.numeric(linear_regression$residuals)
std.residuals <- residuals/sqrt(1.-hatvalues(linear_regression))

par(mfcol=c(1,1))

model_name="CAPM"
hist(std.residuals,
     freq=FALSE,
     nclass=100,
     main=paste(
       c(
         model_name,
         ": Histogram of Std. Residuals \n",
         "Normal Fits: MLE(Green) and Robust(Blue)"),
       collapse="")
     )

std.residuals.mean <- mean(std.residuals)

std.residuals.stdev.mle <- sqrt(mean(std.residuals^2) - (mean(std.residuals))^2)

std.residuals.stdev.robust <- IQR(std.residuals)/1.3490

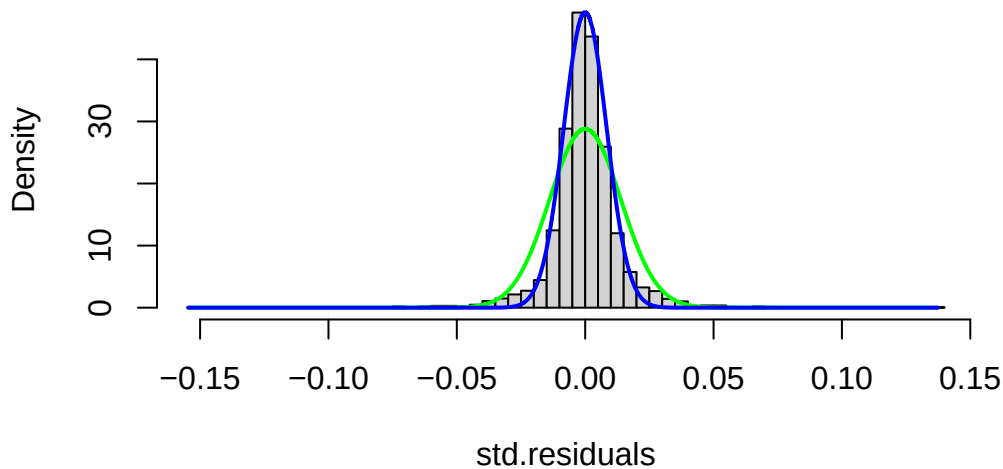
std.residuals.sorted <- sort(std.residuals)

std.residuals.density.mle <- dnorm(std.residuals.sorted,
                                   mean=std.residuals.mean,
                                   sd=std.residuals.stdev.mle)
```

```
std.residuals.density.robust <- dnorm(std.residuals.sorted,
                                     mean=std.residuals.mean,
                                     sd=std.residuals.stdev.robust)

lines(std.residuals.sorted, std.residuals.density.mle, col='green', lwd=2)
lines(std.residuals.sorted, std.residuals.density.robust, col='blue', lwd=2)
```

CAPM: Histogram of Std. Residuals Normal Fits: MLE(Green) and Robust(Blue)



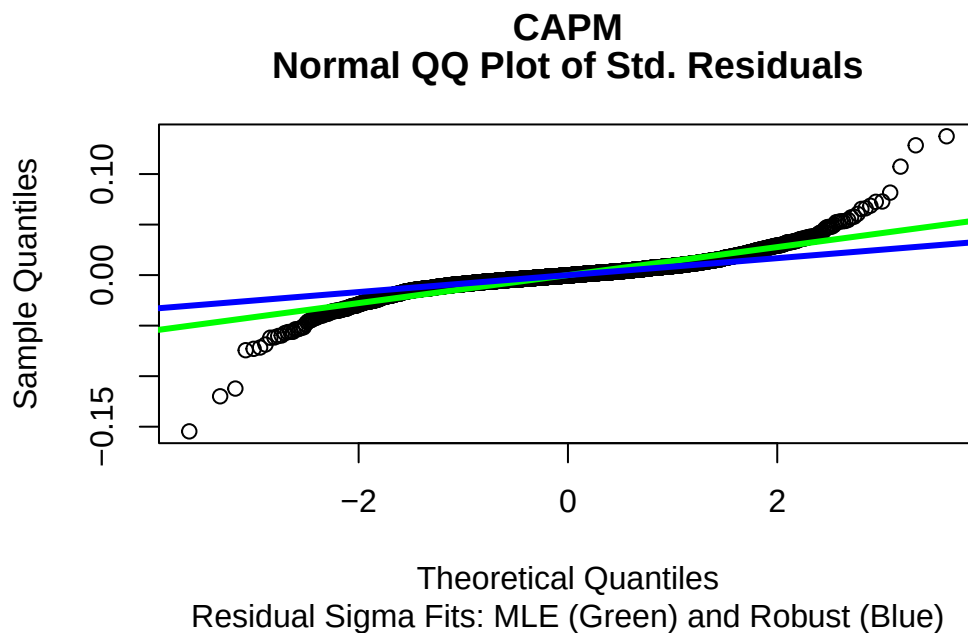
This histogram includes two fitted densities. The first (in green) is specified by the maximum-likelihood estimates (MLEs) of the error mean and variance, assuming the regression residuals are independent and identically distributed Gaussian. In fact, the regression residuals are standardized to have equal variance- their unadjusted variance changes depending on the leverage of cases.

The second density is specified robustly matching the error standard deviation to the observed inter-quantile range ($\hat{\sigma} = \frac{IQR}{1.3490}$).

It appears that the MLE-fitted density under-estimates the distribution near the center, while the robust-fitted density under-estimates the distribution far from the center.

A second graphical method for evaluating consistency with the Gaussian assumption is the QQ-Plot. The residuals are ordered from smallest to largest and these are plotted against the expected values of an ordered sample from a standard Normal distribution (with mean 0 and variance 1).


```
qqnorm(std.residuals,
      main=paste(
        c(
          model_name,
          "\n Normal QQ Plot of Std. Residuals \n"
        ),
        sep=""
      )
)
abline(a=0, b=sqrt(var(std.residuals)), col="green",lwd=3)
abline(a=0, b=IQR(std.residuals)/1.349, col="blue",lwd=3)
title(sub=paste("Residual Sigma Fits: MLE (Green) and Robust (Blue)"))
```



If the data follow a Normal distribution, then the plot should conform to a straight line where the slope of the line is the Gaussian standard deviation and the intercept is the Gaussian mean. In this case, the intercepts are both zero, and the MLE fit has a higher standard deviation than the robust fit.

A third way to evaluate the consistency of the regression-model residuals with a Gaussian distribution is to compute Fitted Percentiles of the sample data and compare the distribution with the uniform distribution. Under either the MLE or robust fit of the Gaussian distribution, we can compute the percentile associated with each residual:

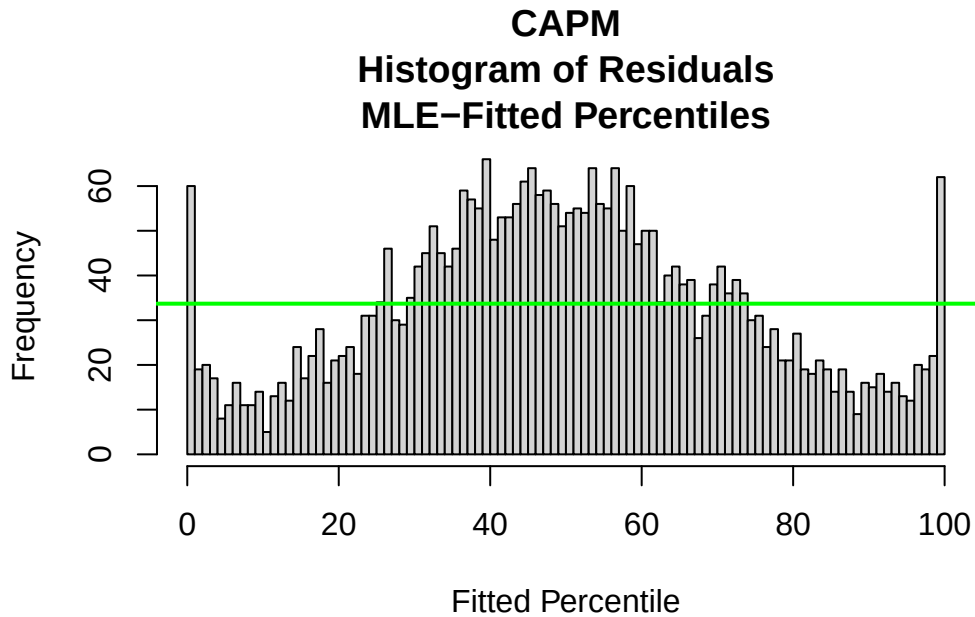
$$\text{Percentile}_i = \Phi^{-1} \left(\frac{\hat{\epsilon}_i - \hat{\mu}}{\hat{\sigma}} \right)$$

where $\Phi(\cdot)$ is the cumulative distribution function of the standard normal distribution $N(0,1)$ and $\Phi^{-1}(\cdot)$ is its inverse function. If the Normal assumption were true, then we would expect the fitted percentiles to follow a uniform distribution between 0 and 100.

The histograms of the fitted percentiles are now displayed for each case:

```
n_class=100
hist(100*pnorm(std.residuals,
               mean = std.residuals.mean,
               sd=std.residuals.stdev.mle),
     nclass=n_class,
     xlab = "Fitted Percentile",
     main=paste(
       c(
         model_name,
         "\nHistogram of Residuals\n",
         "MLE-Fitted Percentiles"),
       collapse="")
     )

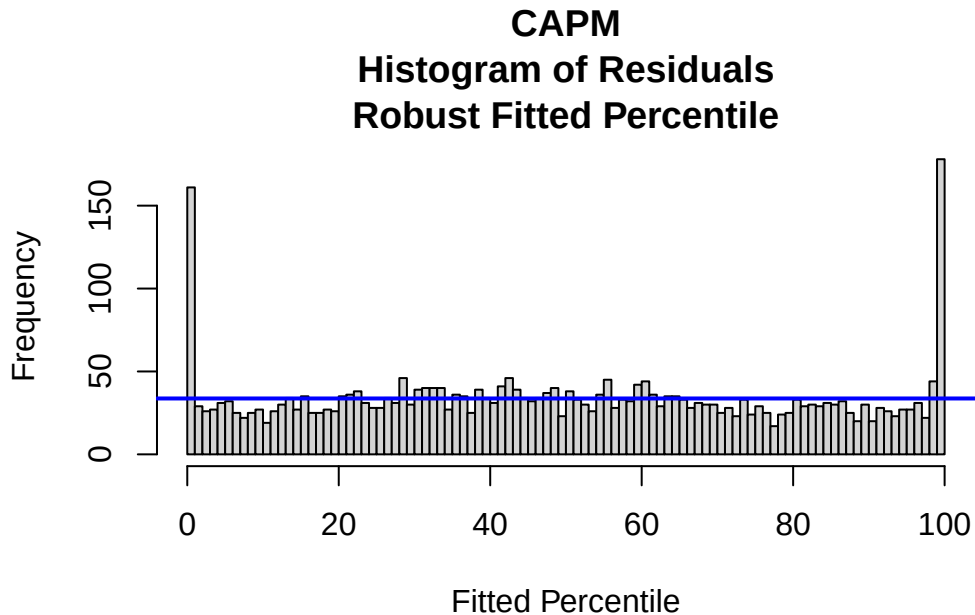
abline(h=length(std.residuals)/n_class, col="green",lwd=2)
```



For the MLE-Fitted Percentiles, the distribution is far from the uniform - there is significant leptokurtosis (slender middle) and heavy tails.

```
n_class=100
hist(100*pnorm(std.residuals,
               mean = std.residuals.mean,
               sd=std.residuals.stdev.robust),
     nclass=n_class,
     xlab = "Fitted Percentile",
     main=paste(
       c(
         model_name,
         "\nHistogram of Residuals\n",
         "Robust Fitted Percentile"),
       collapse="")
     )

abline(h=length(std.residuals)/n_class, col="blue", lwd=2)
```



9 Regression Diagnostics: Influence measures

Some useful R functions

- `influence.measures()`: compute regression diagnostics evaluating case influence for the linear regression model; includes hat matrix, case-deletion statistics for the regression coefficients and for the residual standard deviation.
- `anova()`: conduct an Analysis of Variance for the linear regression model, detailing the computation of the F-statistic for no regression structure.

```
# Compute influence measures (case-deletion statistics)
# measures used to detect influential observations
linear_regression.inflm<-influence.measures(linear_regression)
names(linear_regression.inflm)
```

```
[1] "inflmat" "is.inf" "call"
```

```
# Dimensions of the influence matrix:
# rows = observations, columns = diagnostic statistics
dim(linear_regression.inflm$inflmat)
```

[1] 3370 6

```
# Display rows of the influence matrix
# Columns contain the following diagnostic measures:
#   dfbetas   -> effect of each observation on coefficients
#   dffit     -> effect on fitted values
#   cov.r     -> effect on covariance matrix
#   cook.d    -> Cook's distance
#   hat       -> leverage values
head(linear_regression.inflm$inflmat,2)
```

	dfb.1_	dfb.e_SP	dffit	cov.r	cook.d	hat
2	-0.004642937	-0.0006274413	-0.004683556	1.000853	1.097087e-05	0.0003021588
3	0.015472161	0.0009731732	0.015500288	1.000413	1.201364e-04	0.0002979102

```
# Each column corresponds to a criterion:
# TRUE  -> the observation is flagged as potentially influential
# FALSE -> the observation is not considered influential
#         under that criterion.
#
# Rows correspond to observations in the dataset.
# An observation with several TRUE values may deserve further inspection.
head(linear_regression.inflm$is.inf,2)
```

	dfb.1_	dfb.e_SP	dffit	cov.r	cook.d	hat
2	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
3	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE

```
# Table counts of influential/non-influential cases
# as measured by the hat/leverage statistic.
table(linear_regression.inflm$is.inf[, "hat"])
```

FALSE	TRUE
3241	129

```
# Re-Plot data adding:
#   fitted regression line
#   selective highlighting of influential cases
```

```

# Keep only observations used in the regression
regression_data <- final_dataset %>%
  dplyr::select(excess_SP500, excess_GE) %>%
  na.omit()

title_graph <- paste0(
  "GE vs SP500 Data \n",
  "OLS Fit (Green line)\n",
  "High-Leverage Cases (red points)\n",
  "High Cooks Distance (blue Xs)"
)

plot(
  regression_data$excess_SP500,
  regression_data$excess_GE,
  main = title_graph,
  cex.main = 0.8
)
# Add line x=0 and y=0
abline(h=0,v=0)

# Add fitted regression line
abline(linear_regression, col=3, lwd=3)

# Plot cases with high leverage as red (col=2) "o"s
index.inf.hat<-which(linear_regression.inflm$is.inf[, "hat"]==TRUE)

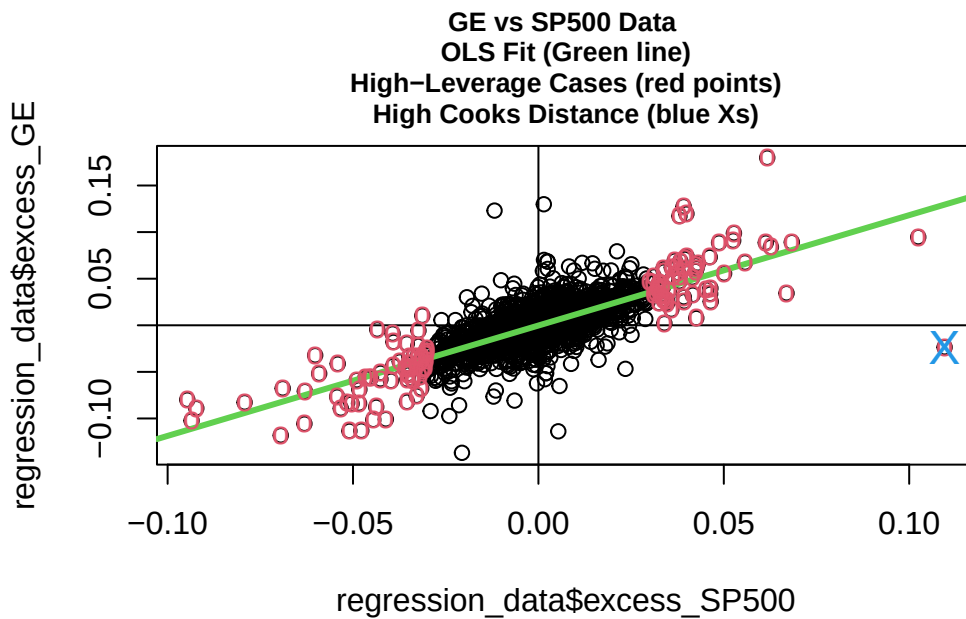
points(
  regression_data$excess_SP500[index.inf.hat],
  regression_data$excess_GE[index.inf.hat],
  col = 2,
  pch = "o"
)

# Plot cases with high cooks distance as big (cex=1.5) blue (col=4) "X"s
index.inf.cook.d<-which(linear_regression.inflm$is.inf[, "cook.d"]==TRUE)

points(
  regression_data$excess_SP500[index.inf.cook.d],
  regression_data$excess_GE[index.inf.cook.d],
  col = 4,
  pch = "X",

```

```
cex = 1.5
)
```

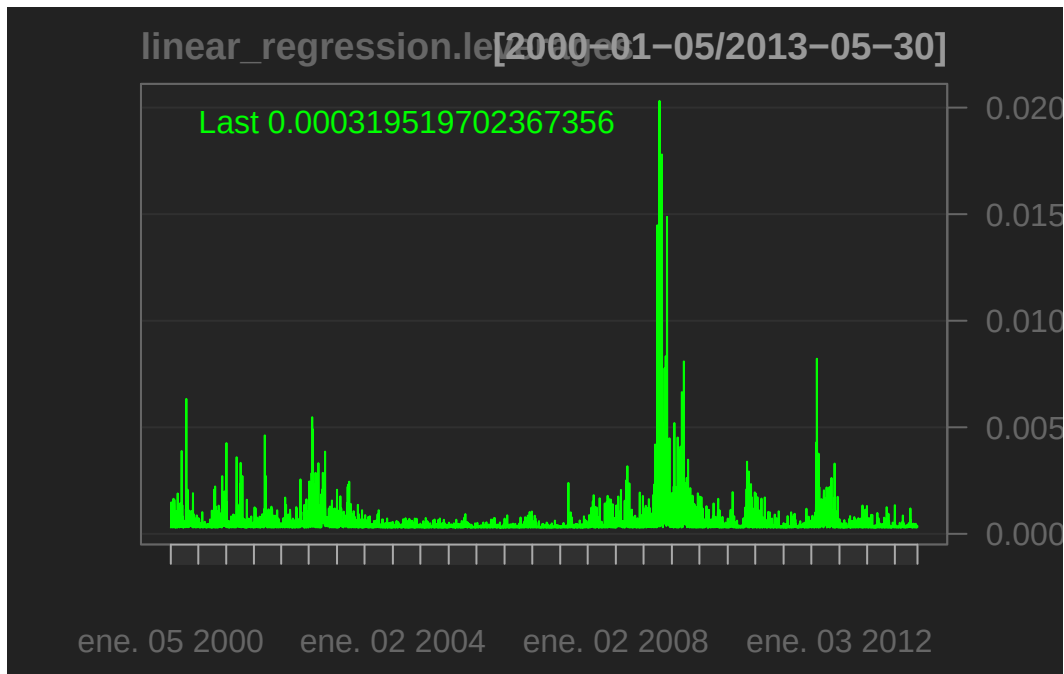


```
# Plot leverage of cases (diagonals of hat matrix)

# Dates corresponding to observations used in regression
regression_dates <- final_dataset$date[complete.cases(
  final_dataset[, c("excess_GE", "excess_SP500")]
)]

# Create xts object containing leverage values indexed by date
linear_regression.leverages <- xts(
  linear_regression.inflm$infmtat[, "hat"],
  order.by = regression_dates
)

# Plot leverage time series
chartSeries(linear_regression.leverages)
```



Note the cases are time points of the time series data:

The financial crisis of 2008 is evident.

The R function `plot.lm()` generates a useful 2x2 display of plots for various regression diagnostic statistics:

```
layout(matrix(c(1,2,3,4),2,2)) # optional 4 graphs/page
plot(linear_regression)
```



```
# The linear regression for the simple CAPM:
linear_regression <- lm(excess_GE ~ excess_SP500, data = final_dataset)
summary.lm(linear_regression)
```

Call:

```
lm(formula = excess_GE ~ excess_SP500, data = final_dataset)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.153047	-0.005734	-0.000304	0.005575	0.137353

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.0002540	0.0002385	-1.065	0.287
excess_SP500	1.1843741	0.0178781	66.247	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.01385 on 3368 degrees of freedom
(1 observation deleted due to missingness)

Multiple R-squared: 0.5658, Adjusted R-squared: 0.5657

F-statistic: 4389 on 1 and 3368 DF, p-value: < 2.2e-16

10.2 Linear regression for the extended CAPM

```
# The linear regression for the extended CAPM:
linear_regression_2 <- lm(excess_GE ~ excess_SP500 + excess_DCOILWTIC0,
                        data = final_dataset
                        )
summary.lm(linear_regression_2)
```

Call:

```
lm(formula = excess_GE ~ excess_SP500 + excess_DCOILWTIC0, data = final_dataset)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.152803	-0.005560	-0.000253	0.005657	0.133479

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-0.0002426	0.0002381	-1.019	0.30824
excess_SP500	1.1976306	0.0181873	65.850	< 2e-16 ***
excess_DCOILWTIC0	-0.0363231	0.0096481	-3.765	0.00017 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.01382 on 3367 degrees of freedom

(1 observation deleted due to missingness)

Multiple R-squared: 0.5676, Adjusted R-squared: 0.5674

F-statistic: 2210 on 2 and 3367 DF, p-value: < 2.2e-16

The regression coefficient for the oil factor is statistically significant and negative. Therefore over the analysis period, price changes in GE stock are negatively related to the price changes in oil.

11 Analysis of Variance

```
# Display the ANOVA table associated with the linear regression model.
#
# ANOVA (Analysis of Variance) decomposes the total variability
# of the dependent variable into:
#
# 1. Variability explained by the regression model
# 2. Residual variability (unexplained variation / errors)
#
# The table typically includes:
#
# Df      -> degrees of freedom
# Sum Sq  -> sum of squares associated with each source of variation
# Mean Sq -> sum of squares divided by corresponding degrees of freedom
# F value -> test statistic for overall model significance
# Pr(>F)  -> p-value associated with the F-test
# Display the ANOVA table associated with the linear regression model.
#
# ANOVA (Analysis of Variance) decomposes the total variability
# of the dependent variable into:
#
```

```
# 1. Variability explained by the regression model
# 2. Residual variability (unexplained variation / errors)
#
# The table typically includes:
#
# Df      -> degrees of freedom
# Sum Sq  -> sum of squares associated with each source of variation
# Mean Sq -> sum of squares divided by corresponding degrees of freedom
# F value -> test statistic for overall model significance
# Pr(>F)  -> p-value associated with the F-test
#
# A small p-value indicates that the regression model explains
# a statistically significant portion of the variation in GE returns.
anova(linear_regression)
```

Analysis of Variance Table

Response: excess_GE

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
excess_SP500	1	0.84142	0.84142	4388.7	< 2.2e-16 ***
Residuals	3368	0.64573	0.00019		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
anova(linear_regression_2)
```

Analysis of Variance Table

Response: excess_GE

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
excess_SP500	1	0.84142	0.84142	4405.875	< 2.2e-16 ***
excess_DCOILWTIC0	1	0.00271	0.00271	14.174	0.0001695 ***
Residuals	3367	0.64302	0.00019		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

11.1 ANOVA Comparison of Nested Linear Regression Models

- Requirements:

The ANOVA comparison of linear regression models requires the models to be **nested**. This means that the larger model must contain all explanatory variables from the smaller model, plus one or more additional variables. For example:

```
simple_model <- lm(Y ~ X1, data = df)
complex_model <- lm(Y ~ X1 + X2, data = df)
```

Here, `simple_model` is nested within `complex_model` because the complex model includes all variables from the first model.

- Objective:

The ANOVA comparison evaluates whether the more complex model provides a statistically significant improvement in fit relative to the simpler model.

The test compares:

RSS_{simple} vs RSS_{complex}

where RSS denotes the Residual Sum of Squares.

- Interpretation:

A small p-value indicates that the more complex model explains a significantly larger portion of the variability in the response variable.

```
anova(linear_regression, linear_regression_2)
```

Analysis of Variance Table

```
Model 1: excess_GE ~ excess_SP500
Model 2: excess_GE ~ excess_SP500 + excess_DCOILWTICO
  Res.Df    RSS Df Sum of Sq    F    Pr(>F)
1    3368 0.64573
2    3367 0.64302  1 0.0027069 14.174 0.0001695 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

ANOVA Output Table contains:

Column	Meaning
Res.Df	Residual degrees of freedom
RSS	Residual Sum of Squares

Column	Meaning
Df	Number of additional parameters introduced in the larger model
Sum of Sq	Reduction in RSS obtained by the complex model
F	F-statistic value used for the nested model comparison
Pr(>F)	p-value associated with the F-test

A small p-value ($\text{Pr}(>F) < 0.05$) indicates that the more complex model explains a significantly larger portion of the variability in the response variable.

12 Exxon-Mobil stock

Consider the corresponding models for the oil stock XOM (Exxon-Mobil)

```
# The linear regression for the simple CAPM:
exxon_mobil <- lm(excess_XOM ~ excess_SP500, data = final_dataset)
summary.lm(exxon_mobil)
```

Call:

```
lm(formula = excess_XOM ~ excess_SP500, data = final_dataset)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-0.085375 -0.005796 -0.000008  0.006226  0.113807
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  0.0002041  0.0002109   0.968   0.333
excess_SP500  0.8324400  0.0158045  52.671 <2e-16 ***
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 0.01224 on 3368 degrees of freedom

(1 observation deleted due to missingness)

Multiple R-squared: 0.4517, Adjusted R-squared: 0.4515

F-statistic: 2774 on 1 and 3368 DF, p-value: < 2.2e-16

```
# The linear regression for the extended CAPM:
exxon_mobil_plus_oil <- lm(excess_XOM ~ excess_SP500 + excess_DCOILWTICO,
                           data = final_dataset
                           )
summary.lm(exxon_mobil_plus_oil)
```

Call:

```
lm(formula = excess_XOM ~ excess_SP500 + excess_DCOILWTICO, data = final_dataset)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-0.086025	-0.005586	-0.000006	0.005805	0.105784

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.0001627	0.0002033	0.80	0.424
excess_SP500	0.7842980	0.0155314	50.50	<2e-16 ***
excess_DCOILWTICO	0.1319092	0.0082391	16.01	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.0118 on 3367 degrees of freedom

(1 observation deleted due to missingness)

Multiple R-squared: 0.4905, Adjusted R-squared: 0.4902

F-statistic: 1620 on 2 and 3367 DF, p-value: < 2.2e-16

Results:

- The R-squared for *XOM* is lower than for *GE*.
- The beta of *XOM* with the SP500 is less than 1.
- The regression coefficient for the oil factor (*DCOILWTICO*) is statistically significant and positive.

For the extended model, we use the R function `plot.lm()` to display regression diagnostic statistics:

```
layout(matrix(c(1,2,3,4),2,2)) # optional 4 graphs/page
plot(exxon_mobil_plus_oil)
```



```
head(exxon_mobil_plus_oil$x)
```

	(Intercept)	excess_SP500	excess_DCOILWTICO
2	1	0.0017692383	-0.0364028287
3	1	0.0008049553	0.0055131791
4	1	0.0265805165	-0.0001494333
5	1	0.0106762602	-0.0036838907
6	1	-0.0132993987	0.0387429692
7	1	-0.0045473971	0.0233157506

We now compute the leverage (and other influence measures) with the function `influence.measures()` and display the scatter plot of the independent variables, highlighting the high-leverage cases.

```
exxon_mobil_plus_oil.inflm<-influence.measures(exxon_mobil_plus_oil)
index.inf.hat<-which(exxon_mobil_plus_oil.inflm$is.inf[, "hat"]==TRUE)

par(mfcol=c(1,1))

plot(exxon_mobil_plus_oil$x[,2],
      Exxon_mobil_plus_oil$x[,3],
      xlab="excess_SP500",
      ylab="excess_DCOILWTICO"
      )
title_graph <- paste0(
  "Scatter Plot of Independent Variables \n",
  " High Leverage Points shown in red"
)
title(main=title_graph)

points(exxon_mobil_plus_oil$x[index.inf.hat,2],
        Exxon_mobil_plus_oil$x[index.inf.hat,3],
        col=2,
        pch="o")
```

Scatter Plot of Independent Variables
High Leverage Points shown in red

